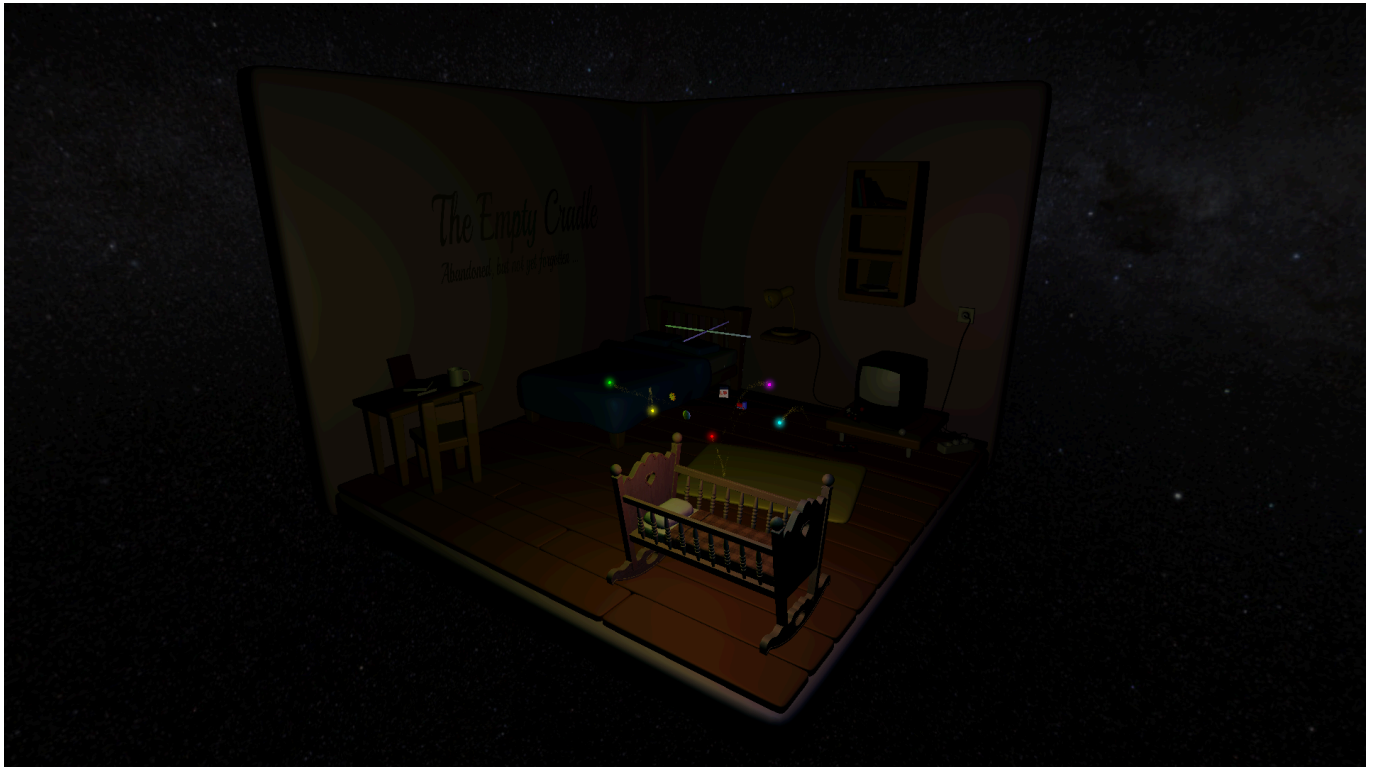


GPU Programming and Real-Time Rendering

Timothy Clarke
University of Dundee
United Kingdom
2712139@dundee.ac.uk



1. Introduction

1.1. The Empty Cradle

Abandoned, but not yet forgotten ...

The fantasy scene shows an empty cradle, abandoned in the corner of a room, but not yet forgotten by the glowing orbs that float above and around it as they drop magical dust where they wander. The cradle gently rocks, and the mobile continues to rotate above. The ornaments naturally spin back and forth, and the train's wheels turn.

The scene is intentionally dark, to show the lighting effect of the orbs. However a stronger light can be toggled on and off using the **L** key, and moved around the scene using the **keypad** (see Section 2). A white sphere represents its location.

An adventure torch is also available, and can be toggled on and off using the **T** key. Coloured lens filters can be used by scrolling the mouse-wheel, and it's beam width adjusted using the **[** and **]** keys.

The cot mobile is made from geometric shapes. The remainder of the scene is loaded from **.obj** files. The cradle and golf ball (next to the television) use diffuse and normal textures from the **.obj** files, while the rest of the loaded scene uses diffuse colours from the files.

The scene uses graph structures, which includes transformation, animation, and particle emission functionality.

The main OpenGL techniques added to the second assignment include:

1. skybox
2. particle emission — instanced, with randomised attributes which adjust according to each particle's time-to-live
3. model loading — using ASSIMP

4. material loading — using ASSIMP
5. normal mapping — using tangents and bitangents
6. tone mapping — using Reinhard, ACES [1], or Uncharted2 [2] algorithms
7. bloom effect — using Optimised 3-, 5-, 7-, or 9-Step Kernel weights

A complete list of features is available in Section 2.3, with new features labelled appropriately.

1.2. What More Would I Have Liked To Do

I tried to add the scene title to the wall as a texture, using Blender. However my laptop is eleven years old, and Blender runs out of memory if I use it too long or try to do too much. As I was unable to do this, I instead used a decal mesh object that I created for the mobile star ornament, and transformed it in the scene appropriately, showing only one of its sides.

The particles were emitted in clumps, due to the time delta between frames. I managed to mostly prevent this by interpolating the orb's change in location and breaking up the time delta. I also added some randomisation to each particle's y-coordinate. Sometimes a clump is visible, though this may be a natural clump formed by the orb's change of direction.

I would have liked to have done more, but did not have the time:

- made some improvements to the **glo** library that I created
- implement shadows
- use name tags for my scene graph to identify nodes, and automatically insert them from the **.obj** files, thus allowing more robust graph navigation

2. Instructions

2.1. Build & Run

The application is built using `cmake`. It should be run from the project's root folder.

```
$ cmake -B build
$ make -C build
$ ./assignment-02
```

2.2. Usage

Movement: **WASD** keys for movement. **R** and **F** to move up and down. Move the mouse to change camera rotation.

Torch: **T** to toggle the torch's power, **[** and **]** to tighten and widen the beam. Use the mouse **scroll wheel** to rotate through the different coloured lens filters.

Main Light: **L** switches the main light off and on. The **keypad** numbers **8**, **4**, **6**, and **2** can be used to move the main light along the x and z axis. **9** and **3** will move it up and down the y axis. **Keypad 5** will reset the light to its initial location.

Orbs: **O** pauses the animation of the orbs. **Shift+O** toggles their visibility (and their light) entirely.

Mobile: **M** pauses the rotation of the cot mobile. **P** pauses the rotation of the mobile, the ornaments, and the cradle.

Animation: **Keypad +** and **-** adjusts all animation speeds.

Particles: **E** pauses the particle emission.

Phong Specular Method: **F5** switches between Blinn Phong Specular and Phong Specular.

Bloom: **F6** and **Shift+F6** adjusts bloom level (number of Gaussian blur taps).

Tone Mapping: **F7** and **Shift+F7** cycles between tone mapping techniques.

Normal Mapping: **F8** toggles off and on normal mapping.

Exit: **Escape** exits the program.

2.3. Key Features

1. The cot mobile rotates continuously at a constant speed in one direction on the y-axis.
2. Ornaments rotate “naturally” in alternating directions on the y-axis. Speed is slowed at the end of a rotation, and increased again when the direction has changed.
3. Train wheels rotate continuously at a constant speed in one direction on the z-axis.
4. Primary lighting provided by five floating orbs, which move above ornaments along randomised Bezier curves.
5. Additional light source (switched off by default) provides stronger light.
6. Weak light also provided by the “adventure” torch.
7. The torch's beam may be made wider or tighter.
8. Tighter torch beams have longer reach than wider beams.
9. The torch's colour filters may be changed using the mouse scroll wheel.
10. WASD and RF (up/down) movement, with viewing angle controlled by the mouse.
11. **NEW** Cot mobile rendered in a loaded model of a room. Each mesh in the room is coloured according to the `.obj` file diffuse colour.

12. **NEW** A cradle is located below the cot mobile. This is loaded from a `.obj` file with textures. An animator is used to gently rock the cradle.
13. **NEW** A skybox of the Milky Way is added.
14. **NEW** The orbs have a particle trail. The particles use an instancing method. Each particle has a randomised yellow-gold colour, and a limited life during which time they slowly fall, fade, and get smaller. The starting position is interpolated over the frame's time delta. These can be paused using **E**, causing existing particles to be frozen, and no new particles to be emitted. The length of the trail is long to allow the drop to be clearly visible.
15. **NEW** A Bloom effect is added to the orbs. The effect can be adjust up or down (circular) using **F6** and **Shift-F6** respectively.
16. **NEW** Tone mapping added, using Reinhard, ACES, and Uncharted2 techniques. These can be cycled through, or turned off, using **F7** and **Shift-F7**.
17. **NEW** Normal mapping implemented for loaded meshes, using tangents and bitangents. This can be toggled off and on using **F8**. A golf ball was added to the scene, next to the television, as this shows the normal mapping more clearly — especially by moving the main light to each side of it.

3. Decision Rationale

3.1. Scene Graphs

The `SceneNode` class can be used to create a graph of `Renderable` objects. Each node may optionally have a `Renderable`, and zero or more child `SceneNodes`. An empty node is valid, yet pointless. Using a graph provides easy management of mesh construction, storage, and destruction. And provides efficient transformation, animation, and particle emission.

Primarily, the graph allows a transformation in a parent node, to be reflected in all descendant nodes, without setting each separately — translating, scaling, and/or rotating a parent, will provide a transformation matrix for each descendant with the same transformations. The transformations are one way, from parent to child, allowing further transformations in the children without affecting parents, cousins, etc.

Methods are provided to allow relative and absolute transformations (e.g. `rotate` and `setRotation`), and access to each transformation (e.g. `getRotation`).

Each `SceneNode` may have zero or more animations. These animations are updated after every call to `animate`. An animation should update the `SceneNode`'s transformation according to its purpose, taking advantage of the transformation mechanics described above.

Each `SceneNode` may also have one or more particle emitters. These are updated after every call to `emit`.

By using shared pointers, the `SceneNode` graph also allows an easy way to store all scene objects. The client may choose to keep its own shared pointer of a node for later access, or access it from the graph using the `getChildren` accessor.

Rendering a `SceneNode` does *not* render the children. Graph traversal should be done manually, to allow shader uniform values to be updated appropriately. A call to `updateTransformation` should be made once, to the top parent node, before the rendering loop begins. This allows any nodes marked as dirty to update their transformation matrix just once. Rendering a mesh is done by a call to `SceneNode.render`. Rendering particles should be done after all render calls are complete, using `SceneNode.renderParticles`.

3.2. Triangle Arrays

My own research into OpenGL practices has led me to understand that modern GPU architectures are optimised for triangle arrays. By taking advantage of this, we no longer have to specify degenerate triangles, and only need to make one call for the whole sphere to the GPU, instead of one call per ring. This all comes at the cost of a higher number of vertices. However, in modern GPUs, space is no longer the premium that it once was.

4. Source Materials

In addition to attending lectures, I also used Learn OpenGL [3] and OpenGL Programming Guide [4] to learn in my own time.

4.1. `glo/camera.*`

I asked DeepSeek [5] to generate code for the `lookAt` method, which changes the camera's yaw and pitch according to the direction of the target location. Apart from modifying the code to fit into the `Camera` class, and adding a clamp, no other modification was required.

4.2. `glo/mesh/shapes/cylinder.*`

Although I have written this class myself, to aid in improving my usage of `sin` and `cos`, I did at times refer to how `SphereMesh` and `StarMesh` work, which themselves are modifications of DeepSeek generated code.

4.3. `glo/mesh/shapes/sphere.*`

I asked DeepSeek to generate code for a sphere, using a triangle array. The code was modified to cater for:

- efficiency — the code was cleaned up, duplication removed, and value calculations minimised (for example, `y` only has to be calculated once per ring).
- types — the data types were changed to use OpenGL types (for example, from `float` to `GLfloat`).
- winding — winding had to be corrected.
- adaption — the relevant code was extracted to place into the `SphereMesh`. Variable names were modified for consistency with the current codebase, and `consts` specified as appropriate.

4.4. `glo/mesh/shapes/star.*`

I asked DeepSeek to generate code for several variations of cogs, as I was considering project ideas and investigating the possibility of a bronze clockwork mechanism. It had previously occurred to me that the geometric shapes, with the test wraps I was using at the time, reminded me of a child's play room. I realised that I could create a star mesh from similar code that DeepSeek had generated for one of the cogs. And so though the generated code has been heavily modified to suit the new shape, the original was generated by DeepSeek. The method `generateEdge` remains mostly intact to what was generated, though winding did need to be fixed. Overall, code was modified to cater for:

- efficiency — the code was cleaned up, duplication removed, and value calculations minimised. In particular, instead of four loops to generate the front and back faces, two of the loops were put into a method, and adapted to generate either face according to the `z` coordinate argument.
- types — the data types were changed to use OpenGL types (for example, from `float` to `GLfloat`).
- winding — winding had to be corrected.
- adaption — the relevant code was modified to create a star instead of a cog, and extracted to place into the `StarMesh`. Variable names were modified for consistency with the current codebase, and `const` specified as appropriate.

4.5. `glo/scene/animators/orb.*`

I asked DeepSeek to generate code for the Bezier curves. I have experience in using Bezier curves with computer art programs, but no prior knowledge of how they work. Therefore, apart from fitting

the code into the animator, the Bezier curve calculation is fully generated by DeepSeek.

4.6. `scene/graphs/light_properties.h`

The Ogre3d wiki [6] was used for linear and quadratic values for various light ranges.

I asked DeepSeek to extrapolate linear and quadratic values from the data for ranges that I felt more desirable.

4.7. `scene/torch.*`

I had a discussion with DeepSeek to find suitable properties for an "adventure torch". This included colours for the plastic filters, and a colour for a bare incandescent bulb. I considered the beam width and attenuation suggestions too large, so asked for pen light properties and used those values instead.

4.8. `textures/*.png`

I made several prompts and refinements to Gemini AI [7] to generate images for the sphere (`animals.png`), cube (`letters-cube.png`), and star (`sun.png`). Each image was requested separately (i.e. two images for the sphere, six for the cube, and two for the star). These images were modified as needed using Inkscape and Gimp, and added to appropriate texture wraps.

4.9. `textures/skybox/*.jpg`

I sourced DDS image files from <https://spacedock.info/mod/926/Pood/%27s/%20Milky%20Way/%20Skybox> and converted them to JPEG files using Gimp.

4.10. `glo/scene/emitters/trail.*`

I was already intending to drop the particles, and reduce their size, over time. The idea of fading them came from reading about particles in the Learn OpenGL guide, where I also learned of the `GL_ONE` blend function. I thought randomising the colour a little would also add some "sparkle" and asked DeepSeek to give me a range of individual RGB floats that added together would create a yellow-gold colour.

I was struggling with the particles turning invisible at certain camera angles. I reread the Learn OpenGL Guide sections for both particles and instancing. I learned about Billboarding — I had missed its importance previously. I couldn't see where the guide covered this technique in detail, so used DeepSeek to help me understand the concept.

I ended up with a combination of instancing and particles. Because I had been reading both sections from the Learn OpenGL Guide, I thought that my implementation might be wrong, and that I should be using one method or the other (the two sections are separated by 239 pages in the PDF). I searched the internet, and realised that this is considered the best approach. I could have used just particles, as I am only rendering circles, but by using instancing I have more flexibility should I choose to, for example, add a texture or change the shape.

4.11. `shaders/blur.frag`

I originally took the code from the Learn OpenGL guide. It is a fixed algorithm, but I thought the original code a little messy and difficult to maintain. So I tidied it up by adding appropriate variable usage to make the process clearer, and added comments, all so I could clearly understand the algorithm myself — which as it became clear, is very simple.

The original weights that I used also come from the Learn OpenGL guide. I did some investigation into different Gaussian approaches. From what I learned, the single-pass approach produces similar quality results, but is less efficient (sampling N^2 pixels for each pass, compared to $2N$ pixels for the two-pass approach ($4N$ total)).

I also learned that there are, for example, 5-Tap Kernel and Optimised 5-Tap Kernel sets of weights — the optimised versions use computationally cheaper numbers, with a negligible drop in quality.

I experimented with 3-, 5-, 7-, and 9-Tap weights, which I asked DeepSeek for. I felt the 9-Tap weights offered a significant quality improvement which outweighed the extra computational cost over the alternatives that I tried.

However, I was keen to allow the amount of bloom to be adjusted via user input, and so I rewrote the code entirely to remove the horizontal/vertical duplication (by introducing the direction variable), and worked around the nature of GLSL's array handling by introducing switch statements. This sacrifices a little performance for improved flexibility and maintainability.

4.12. shaders/bright_pass.frag

I found the luminance vec3 in the Learn OpenGL guide, but did some research into its meaning and original source. It originates from the HDTV Recommendation ITU-R BT.709-6 standard for HDTV [8].

The Learn OpenGL Guide provides the Reinhard tone mapping technique. I wanted to know what else was available, and found that ACES and Uncharted2 are both popular techniques. These can all be cycled through using the F6 key.

4.13. Models

I purchased the isometric bedroom model [9] from cgtrader [10]. The cradle and its textures [11] also come from cgtrader. The golf ball and its textures [12] came as a free download from Sketchfab [13].

5. UML

A simplified UML diagram has been made for the source code. It can be found in the document assignment-02-uml.pdf.

References

- [1] K. Narkowicz, "ACES Filmic Tone Mapping Curve." Accessed: Nov. 29, 2025. [Online]. Available: <https://knarkowicz.wordpress.com/2016/01/06/aces-filmic-tone-mapping-curve/>
- [2] "Filmic Tonemapping Operators." Accessed: Nov. 29, 2025. [Online]. Available: <http://filmicworlds.com/blog/filmic-tonemapping-operators/>
- [3] J. de Vries, *Learn OpenGL: Learn Modern OpenGL Graphics Programming in a Step-by-Step Fashion*. Erscheinungsort nicht ermittelbar: Kendall & Wells, 2020.
- [4] J. M. Kessenich, G. Sellers, and D. Shreiner, *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.5 with SPIR-V*, Ninth edition. Boston Columbus Indianapolis New York San Francisco Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal Toronto Delhi Mexico City São Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo: Addison-Wesley, 2017.
- [5] "DeepSeek." Accessed: Nov. 11, 2025. [Online]. Available: <https://chat.deepseek.com/>
- [6] "Point Light Attenuation." [Online]. Available: <http://wiki.ogre3d.org/0/-Point+Light+Attenuation>
- [7] "Google Gemini." Accessed: Nov. 11, 2025. [Online]. Available: <https://gemini.google.com/>
- [8] "BT.709~::~Parameter Values for the HDTV Standards for Production and International Programme Exchange." Accessed: Nov. 29, 2025. [Online]. Available: <https://www.itu.int/rec/r-rec-bt.709>
- [9] "Isometric Bedroom | 3D Model." Accessed: Nov. 30, 2025. [Online]. Available: <https://www.cgtrader.com/3d-models/interior/bedroom/isometric-bedroom-e4d0471d-e743-4b45-b432-225e72bda783>
- [10] "CGTrader - 3D Model Store." Accessed: Nov. 30, 2025. [Online]. Available: <https://www.cgtrader.com/>
- [11] "Stylized Cartoon Cradle - 2 Color Schemes | 3D Model." Accessed: Nov. 30, 2025. [Online]. Available: <https://www.cgtrader.com/3d-models/furniture/bed/low-poly-cartoon-cradle>
- [12] "Golf Ball Low-poly - Download Free 3D Model by MaX3Dd." Accessed: Nov. 30, 2025. [Online]. Available: <https://sketchfab.com/models/bd117023914545ac83382c303b50350b/embed?autostart=1>
- [13] "Sketchfab." Accessed: Nov. 30, 2025. [Online]. Available: <https://sketchfab.com/feed>